VILORA

BACKEND TEAM WORKFLOW — V1.0

Two-member execution plan • Developer Alpha / Developer Beta • V1 only

1. WORKING RULE

Build one stable V1 first. Do not create or discuss a Beta version. Future improvements are tracked as V1.x or a later V2.0 only after V1 is complete.

2. DEVELOPMENT SEQUENCE

Step	Work	Status
01	Project + environment setup	Both
02	Database + common entities/relationships	Both
03	Common configuration + exception handling + validation base	Both
04	Authentication + authorization foundation	Alpha
05	Catalog: categories + products	Beta
06	Cart + wishlist	Alpha
07	Addresses + users	Beta
08	Orders	Alpha
09	Payments	Beta
10	Reviews	Alpha
11	Frontend API integration	Both
12	Swagger + testing + security review	Both
13	Integration + bug fixing	Both
14	V1.0 release	Both

3. DEVELOPER OWNERSHIP

Developer Alpha

AUTH → CART → WISHLIST → ORDERS → REVIEWS

Developer Beta

CATEGORIES → PRODUCTS → ADDRESSES → USERS → PAYMENTS

Shared

Initial setup → database contract → common configuration → integration → testing → Swagger → final V1 review.

4. FOUNDATION — DO THIS BEFORE PARALLEL DEVELOPMENT

01. 01. Create Spring Boot project and confirm Maven build.

02. 02. Configure application profiles / environment variables.

03. 03. Configure PostgreSQL.

04. 04. Configure JPA + Hibernate.

05. 05. Configure Lombok.

06. 06. Configure Spring Security + JWT dependencies.

07. 07. Create package structure: config / controller / dtoHelper / entity / exception / persistence / pojo / repository / service.

08. 08. Define database entities and relationships before implementing module APIs.

09. 09. Define API response/error format.

10. 10. Define common validation, exception and pagination conventions.

11. 11. Commit foundation and tag the baseline before parallel work.

5. API IMPLEMENTATION RULE FOR EVERY MODULE

01. Entity → Repository → DTO → Service → Controller → Validation → Exception handling → Test → Swagger.

02. Use annotations/Lombok wherever appropriate; keep business logic in the service layer.

03. Do not expose JPA entities directly when a DTO is appropriate.

04. Use pagination/sorting on collection endpoints where the dataset can grow.

05. Use consistent HTTP status codes and error responses.

6. MODULE CHECKLIST

AUTH

Register • Login • Me • Password/credential validation • JWT • Role checks

CATEGORIES

List • Get by ID • Create • Update • Delete • Validation

PRODUCTS

List + pagination • Filtering • Sorting • Get by ID • Create • Update • Delete • Stock/availability

CART

Get cart • Add item • Update quantity • Remove item • Clear cart • Stock/quantity validation

WISHLIST

Get wishlist • Add product • Remove product • Duplicate prevention

ADDRESSES

List • Get by ID • Create • Update • Delete • Validation

USERS

Get current user • Update current user • Authorization

ORDERS

Create order • List user orders + pagination • Get order • Cancel order • Price/stock snapshot

PAYMENTS

Create payment • Verify payment • Get payment • Signature/status validation

REVIEWS

List product reviews + pagination • Create • Update own review • Delete own review

7. GIT / COORDINATION WORKFLOW

01. main → stable V1 branch.

02. Each developer works only on their assigned feature branch.

03. Never develop directly on main.

04. Before starting work: pull/rebase latest shared changes.

05. One logical feature per branch.

06. Commit in small, meaningful units.

07. Open PR → review → resolve conflicts → merge.

08. After merge, both developers sync before starting the next dependent module.

09. Shared files require coordination before editing.

8. FRONTEND ↔ BACKEND INTEGRATION FLOW

01. Freeze endpoint contract before frontend integration.

02. Share base URL + endpoint + request/response JSON + authentication requirement.

03. Frontend replaces mock/static data with API calls.

04. Test loading, success, empty, validation-error, unauthorized and server-error states.

05. Test real user flow: Register/Login → Browse → Product → Cart → Address → Order → Payment → Confirmation.

06. Fix contract mismatches immediately; do not silently change API structure.

9. V1 QUALITY GATE

01. All V1 endpoints implemented and documented.

02. Authentication and authorization verified.

03. Validation and global exception handling verified.

04. Pagination + sorting implemented where applicable.

05. Database relationships tested.

06. No business logic in controllers.

07. No hard-coded secrets.

08. Swagger/OpenAPI updated.

09. Unit/integration/API tests for critical flows.

10. Frontend end-to-end flow works against the real backend.

10. VERSIONING RULE

V1.0 = first complete, stable production-ready scope defined for Vilora.

V1.1 / V1.2 = bug fixes and small backward-compatible improvements.

V2.0 = only when there is a major scope/API/data-model change that justifies a new version.

Until V1.0 is complete, the team works only toward V1.0.